Design and Implementation of a TinyBASU Processor Simulator with Branch Prediction Analysis

Team: auf keinen fall

Bu-Ali Sina University — Department of Computer Engineering

- **Shahriar kolivand(40312358035)**
- **Abolfazl asali(40312358030)**
- **Amin rahimi mehrnia(40312358013)**

Dr.madhi abbasi

Spring 2026

1. Explanation of Functions (simulator.py)

Function	Description
init	Initializes the 8 registers (zeroed), the 512-word memory (zeroed), the PC, the cycle/instruction/stall counters, and the branch prediction table BPT (empty dictionary).
sign_extend	A global helper that sign-extends an n-bit unsigned value to a correctly signed Python integer using an XOR/subtract mask trick.
reg_num	Extracts the numeric index from a register token such as rx3 (→ 3).
parse_instruction	A two-pass assembler: the first pass records the address of every label; the second pass encodes each line via encode into 16-bit machine code and writes it into memory starting at address 0.
_resolve_immediate	Resolves an immediate operand: returns it directly if it is a number, or (addr+1)) if it is a label. converts a label into a PC-relative offset (target
encode	Packs the opcode/rd/rs/rt/func/imm fields into the correct R/I/J format based on the mnemonic; all 12 instructions of the instruction set are covered.
init_memory	Reads the hexadecimal data file line by line and places the values starting at memory address 256.
fetch	Reads the memory word at the current pc, increments pc by one, and returns the raw 16-bit word.
decode	Uses bit-shifting and masking to extract the opcode, rd, rs, rt, func fields and the sign-extended immediate values i_imm/j_imm from the 16-bit word.
execute	Based on the opcode, it performs the actual operation (arithmetic, memory access, or jump) of each of the 12 instructions on the registers/memory.
branch_prediction	Predicts Taken/Not-Taken for a given branch address according to the currently selected method full code in Section 2.
update_branch_prediction run	Once the actual branch outcome is known, updates the BPT state according to the selected method's state machine - full code in Section 2. The main fetch-decode-execute loop; for every beq/bne it predicts and compares against the actual outcome, adds a 3-cycle penalty and increments num_stalls on a misprediction, and continues until the program ends or a timeout occurs.

Function	Description
report	Computes prediction accuracy and speedup from the counters and writes the final report to the output file in the specified format. Note: it does not currently compute IPC see Section 5, Known Limitations.

2. Branch Prediction Functions (Full Code)

Both functions key self. BPT by the address of the branch instruction (not by (opcode, rs, rt)), since the address is the correct, unique identifier of each static branch site.

```python
def branch_prediction(self, branch_addr): method = self.prediction_method
if method == 'ST':
return True
if method == 'SN':
return False
if method == 'D1':
last = self.BPT.get(branch_addr, return last
if method == 'D2':
# always predict Taken
# always predict Not-Taken
#1-bit: last actual outcome
False)
# 2-bit saturating counter (0..3)
state = f.BPT.get(branch_addr, 1) # default: Weakly Not-Taken return state >= 2
if method == 'IQ':
state = self.BPT.get(branch
return state >= 4
# Taken if state is 2 or 3
# 3-bit saturating counter (0..7)
3)
# Taken if state >= 4
raise ValueError(f"Unknown prediction method '{method}'")

def update_branch_prediction (self, branch_addr, actual_taken): method =
self.prediction_method
if method in ('ST', 'SN'):
return
if method == 'D1':
# no table, nothing to update
self.BPT[branch_addr] = actual_taken # just overwrite with the latest outcome
return
if method == 'D2':
state = self.BPT.get(branch_addr, 1)
```

```
state = min(state + 1, 3) if actual_taken else max(state 1,0)
self.BPT[branch_addr] = state
return
if method == 'IQ':
state = self.BPT.get(branch_addr, 3)
state = min(state + 1, 7) if actual_taken else max(state 1,0)
self.BPT[branch_addr] = state
return
```

3. Comparative Analysis of the 15 Reports

Program	Method	Cycles	Executed Instr.	Stalls	Accuracy	IPC*	Speedup
fibo_beq (Fibonacci 30)	ST	254	173	27	4%	0.68	1.01
	SN	176	173	1	96%	0.98	1.46
	D1	176	173	1	96%	0.98	1.46
	D2	176	173	1	96%	0.98	1.46
	IQ	176	173	1	96%	0.98	1.46
fibo_bne (Fibonacci 30)	ST	149	146	1	96%	0.98	1.54
	SN	227	146	27	4%	0.64	1.01
	D1	152	146	2	93%	0.96	1.51
	D2	152	146	2	93%	0.96	1.51
	IQ	152	146	2	93%	0.96	1.51
fact (50! via nested repeated addition)	ST	10791	6822	1323	4%	0.63	1.01
	SN	6972	6822	50	96%	0.98	1.57
	D1	7116	6822	98	93%	0.96	1.54
	D2	6972	6822	50	96%	0.98	1.57
	IQ	6972	6822	50	96%	0.98	1.57

Main observation

— which method wins?

No single algorithm wins across all three programs; the winner is always whichever static predictor happens to align with that particular branch's dominant direction. In fibo_beq and fact , the beq conditions are loop-exit tests and are almost always Not-Taken (Taken only once during the entire run) → SN wins outright. In fibo_bne the opposite holds: bne is the loop-continuation condition and is almost always Taken → ST wins outright. In all three cases, the "correct" static predictor is effectively unbeatable because it is wrong only once in the entire run (at the moment the loop is exited).

Why do the dynamic predictors (D1/D2/IQ) never beat the "correct" static one?

Because these programs exhibit only one behavioral pattern throughout execution (uniform repetition until the end of the loop, then a single exit), there is no oscillation for a dynamic predictor to "learn" from that would give it an edge; the best a dynamic predictor can do is match the correct static predictor, and in the worst case (when the initial default state is misaligned with the program's dominant direction) it incurs one extra "warm-up" misprediction the first time that branch address is encountered.

D1 vs. D2/IQ — the real point of divergence in the data:

In fact , which has two independent branch addresses (an outer loop and an inner loop whose iteration count varies between successive passes of the outer loop), D1 clearly falls behind D2/IQ with 98 stalls and 93% accuracy versus 50 stalls and 96% for D2/IQ; whereas in fibo_beq / fibo_bne , which each have only one branch address per loop, all three are exactly tied. The reason: D1 has no memory beyond "the last outcome," so every time the inner loop's length changes between two consecutive passes of the outer loop, it immediately flips its prediction and mispredicts again; the saturating counters in D2/IQ, however, do not react to a single anomaly and keep 3 / 5 the prediction stable. This is exactly the advantage expected from 2-/3-bit counters on irregular nested loops, and it is what was actually observed in this project's data.

4. Answers to the Analytical Questions

Question 1: Can changing the assembly code (without changing the prediction algorithm) reduce the number of cycles/mispredictions?

Yes, and this project's own data shows it directly. By simply choosing the conditional instruction whose "taken" bias matches the loop's dominant behavior, cycle count can be cut by roughly 40%.

Question 2: Does supporting (or not supporting) data forwarding in the pipeline affect the performance of a prediction method? Why?

Forwarding addresses data hazards (RAW) between pipeline stages and is a separate issue from control hazards. However, without forwarding, the penalty of every misprediction effectively grows larger, so algorithms with fewer mispredictions gain relatively more from the absence of forwarding.

Question 3: What effect does increasing/decreasing the number of pipeline stages have on prediction accuracy and cycle count?

The number of pipeline stages has no direct effect on the algorithm's own accuracy, but it does change the size of the misprediction penalty. The deeper the pipeline, the more practical value a more accurate predictor delivers.